

ORCA — Knowledge Transfer Document

Project: ORCA (Open Retail Command Agent) **Prepared for:** Incoming engineer (eval workstream) **Prepared by:** Ankit (Data Science Manager) — handover before USA travel **Status of project:** Sprints 1–5 complete and deployed live. Sprint 6 (evaluation) in progress. **Your focus:** The **evaluation** workstream (Layer 1 retrieval eval is started; Layers 2 & 3 are yours to build).

Read this document top to bottom **once** before touching the repo. Then keep it open beside your editor. It is long on purpose — by the end you should understand not just *what* ORCA is, but *why* every piece exists. Sections 12–14 are your actual task and the open issues; everything before that is context.

Note: this document also exists as a polished Word pack (ORCA_Onboarding_Pack.docx) with two embedded diagrams (system architecture, and the ESCALATE/HITL flow). If you want those diagrams in the repo, drop the PNGs into docs/img/ and reference them here.

0. How to read this document

Section	What it covers	Priority for you
1–2	What ORCA is and why it exists	Read first
3	The big-picture architecture	Read first
4	The 4 agents and their business logic	Important
5	The data model (SQLite + RAG docs)	Important
6	The RAG pipeline (this is what you’ll evaluate)	Critical
7	File-by-file walkthrough of the repo	Reference
8	Tech stack and glossary	Reference (fresher-friendly)
9	Environment setup — get it running locally	Do this Day 1
10	Deployment (Render)	Read
11	The known landmines (C:/lit, CrewAI, etc.)	Read before debugging anything
12	Your task — the evaluation workstream	This is your job
13	Issue tracker — everything still pending	Critical
14	Glossary	Reference whenever stuck

1. What is ORCA?

ORCA stands for **Open Retail Command Agent**. It is an **autonomous retail inventory management system** built with open-source AI tooling.

In plain terms: it watches inventory across many retail stores, and when a product is about to run out, an AI pipeline automatically figures out *how much to reorder, from which supplier, at what cost, and whether a human needs to approve it before the order is placed*.

It is a **multi-agent system** — four AI “agents” each handle one stage of the decision, passing their work down a chain like an assembly line.

The system is **HITL (Human-In-The-Loop)**: for expensive or risky orders, the pipeline **pauses** and waits for a human to click *Approve* or *Reject* before any money is committed. Cheap, routine orders execute automatically.

2. Why does ORCA exist? (The backstory — important context)

Ankit spent ~10 years in data engineering: Informatica → Palantir Foundry → Palantir AIP Studio. On Palantir he built a real production system called **RCC (Retail Command Centre)** for a **UAE retail client, deployed across 200+ stores**. RCC did exactly what ORCA does — but it was built on **Palantir**, which is a *closed, proprietary platform*.

The problem: skills learned only on a closed platform don’t transfer well to open-source job interviews (Fortune 100 / FAANG AI Engineer roles). So ORCA is a deliberate **open-source rebuild of RCC** — same business problem, but using the open-source agentic AI stack that the industry actually interviews on (LangGraph, RAG, MCP, CrewAI, FastAPI, Docker, etc.).

Why this matters to you: Many design choices in ORCA are “how would the industry do what Palantir did for us automatically?” If something looks over-engineered, it’s usually because Palantir gave it for free and we had to rebuild it by hand. That rebuild *is* the learning.

Key reference points you’ll see repeatedly:

- **RCC** = the original Palantir system (the “source of truth” for business logic).
- **AIP Studio** = Palantir’s agent-building tool (where RCC’s agents lived).
- **Ontology** = Palantir’s data-object layer (ORCA replaces it with a SQLite database + Python query functions).

3. Big-picture architecture

The full flow, end to end:

```
scheduler.py (simulates real-time alerts)
marks SKUs as CRITICAL / AT_RISK in the DB
```

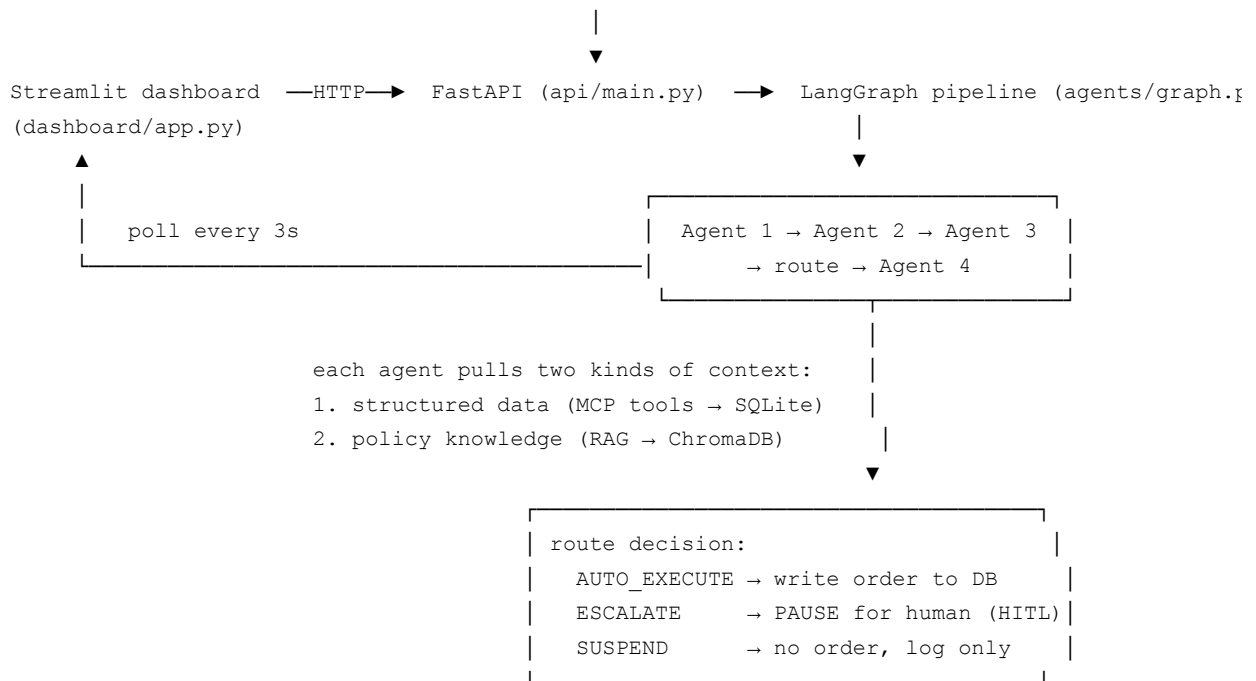


Figure 1 — ORCA system architecture: dashboard to API to the four-agent pipeline, the two context sources, and the three outcomes.

The two context sources are the heart of the system:

1. **MCP tools** fetch *structured facts* from the SQLite database (current stock, supplier lead times, capital budgets).
2. **RAG** fetches *policy knowledge* from text documents (ordering rules, event playbooks, scoring formulas).

You will be **evaluating the RAG half** — measuring whether the right policy knowledge reaches each agent.

4. The four agents (the business logic)

The pipeline is a fixed chain: **Agent 1 → Agent 2 → Agent 3 → routing → Agent 4**. Each agent is one “node” in LangGraph. Each calls the LLM once (Groq `llama-3.1-8b-instant`) with (a) structured data and (b) RAG policy context, and returns a JSON result that flows to the next agent via shared state (`AgentState`).

Agent 1 — Demand Intelligence

Question it answers: *How urgent is this, and will a normal order arrive in time?*

- Classifies **urgency**: `CRITICAL` / `HIGH` / `MEDIUM` (driven mainly by how many stores are critical).
- Computes `lead_time_too_late` — `True` if the supplier’s lead time is longer than the time until an upcoming demand event.
- Estimates projected demand (uses event uplift factors, e.g. Ramadan multiplies demand).
- Also launches a **CrewAI forecasting crew** for a deeper demand read (⚠ currently failing — see Issue #1).

Agent 2 — Supply Replenishment

Question it answers: *What are our reorder options?*

- Builds exactly **three options**:
 - **Option A** — standard reorder (normal lead time).
 - **Option B** — profit-maximising / partial distribution (only top stores).
 - **Option C** — expedite (air freight, faster but expensive).
- **Hard rules** (these are common test targets):
 - **Class A SKUs NEVER get Option B** — they always require full store distribution.
 - **Suppliers with `allows_expedite = No` cannot do Option C** (e.g. Hindustan FMCG). If expedite is needed but not allowed → escalate to a human.

Agent 3 — Capital Allocation

Question it answers: *Which option wins financially, and do we need approval?*

- Scores each option with an **exact formula** (memorise this — it's a frequent eval target):

```
budget_score      = (1 - cost / available_budget) × 40
availability_score = availability_pct × 0.40 × 100
margin_score      = (1 / margin_priority_rank) × 20
lead_time_penalty = -20    IF urgency = CRITICAL AND lead_time_days > 30    (else 0)

total_score = budget_score + availability_score + margin_score + lead_time_penalty
The option with the highest total_score wins.
```

- Determines approval:

```
approval_required = (cost > pool.auto_approve_limit_aed)
```

- A common mistake the agent must avoid (it's in the RAG docs as a “known error”): applying the `lead_time_penalty` when urgency is `HIGH`. It applies **only** when urgency is `CRITICAL`.

Agent 4 — Exception / HITL Briefing

Question it answers: *What happens next, and what does the human need to know?*

- First, a **pure-Python routing decision** (no LLM) picks the route:

```
IF    pool_pressure_flag == HIGH      → SUSPEND      (budget too tight, no order)
ELIF  approval_required == False     → AUTO_EXECUTE (cheap enough, just do it)
ELSE                                     → ESCALATE     (needs a human to approve)
```

- For **ESCALATE**, Agent 4 writes a **briefing** (the text a human planner reads before approving) — including supplier contact, cost, which capital pool, and why approval is needed.
- For **AUTO_EXECUTE**, it writes the order to the DB and logs a confirmation.

HITL mechanism: the LangGraph is compiled with `interrupt_before=["execute_node"]` . This means the graph **physically pauses** before writing any order. A human approves via the dashboard → the API calls `resume_pipeline()` → the graph continues from its saved checkpoint. State is kept by a `MemorySaver` checkpointer keyed by `thread_id = pipeline_id` .

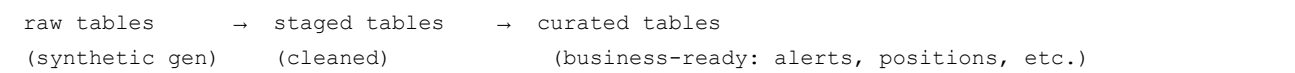
The anchor scenario (memorise this — it’s the canonical demo)

- **ESCALATED example:** *Ajwa Dates* during the *Ramadan* surge. Air-freight (expedite) cost **AED 108,909** exceeds the CP003 pool’s auto-approve limit of **AED 20,000** → pipeline **ESCALATES** and waits for human approval.
- **AUTO_EXECUTED example:** *Dish Soap (SKU00097)* at **AED 817**, under the CP001 limit → executes automatically, no human needed.

5. The data model

5.1 The SQLite database (`db/orca.db`)

Built in three layers (a Palantir habit carried over deliberately):



Conceptually the curated layer holds these object-like tables (this is what the MCP tools query):

- **Inventory positions** — stock level per SKU per store, with a `stock_status` (CRITICAL / AT_RISK / OK) and `reorder_triggered` flag.
- **SKUs** — product master: `sku_id` , `category` , `abc_class` (A/B/C), margin info.
- **Suppliers** — lead times, reliability, `allows_expedite` , `expedite_premium_pct` , `contacts`.
- **Capital pools** — budgets (e.g. CP001, CP003), `auto_approve_limit_aed` , `pool_pressure_flag` .
- **Events** — the retail event calendar with demand uplift %.
- **alerts** — the working list of SKUs needing attention (the dashboard’s main feed).
- **pipeline_log** — audit trail of every completed pipeline run.

The data is **synthetic** (generated by the data scripts) but modelled on the real UAE retail dataset from RCC.

Real reference data you’ll see in tests (from the source Excel files):

Supplier	Category	Lead time	Expedite?	Premium
Al Rawdah Foods	Grocery	30 d	Yes	35%
Emirates Dairy Co	Dairy	5 d	Yes	20%
Gulf Beverages LLC	Beverages	7 d	Yes	25%

Hindustan FMCG	Personal Care	45 d	No	0%
Dragon Imports	Seasonal	60 d	Yes	50%

Event	Uplift	Planning lead	Categories
Ramadan 2025	180%	60 d	Grocery, Dates, Beverages
Eid Al Fitr 2025	220%	45 d	Seasonal, Personal Care, Grocery
Eid Al Adha 2025	150%	45 d	Grocery, Meat, Seasonal
UAE National Day	80%	30 d	Seasonal, Beverages
Back to School	120%	45 d	Stationery, Bags, Electronics

5.2 The RAG documents (docs/)

Five hand-written `.txt` knowledge documents, ingested into ChromaDB as **71 chunks**. These hold the *policy knowledge* the database can't express:

1. `replenishment_policy.txt` — ABC-class ordering rules, stock-status rules, capital-pool rules, lead-time rules, approval routing.
2. `supplier_sla.txt` — all 10 suppliers: lead times, reliability, expedite premiums, contacts.
3. `event_playbook.txt` — all 10 UAE retail events: uplift %, planning lead days, ordering rules.
4. `entity_relationships.txt` — GraphRAG-style: SKU→Supplier→Pool chains, risk chains, scoring-formula context.
5. `agent_reasoning_patterns.txt` — Corrective-RAG style: 4 golden worked examples, 6 common errors, self-correction triggers.

Rule baked into the system: *the database always wins over RAG on any factual conflict*. If the DB says a supplier's email is X and a document says Y, the DB is authoritative. The RAG context string literally starts with a "PRIORITY RULE: database wins" line.

6. The RAG pipeline (what you will evaluate — read carefully)

This is the most important section for your work.

6.1 What RAG is doing here

When an agent needs policy knowledge, it does **not** dump all 5 documents into the prompt. Instead it asks the **retriever** for the most relevant chunks, formatted into a compact context string. That string is injected into the agent's prompt.

6.2 The retrieval stack (in docs/rag/retriever.py)

- **Vector store:** ChromaDB, persisted at `db/chroma/`.

- **Embedding model:** `nomic-ai/nomic-embed-text-v1.5` (primary), falls back to `all-MiniLM-L6-v2`.
- **Reranker:** `BAAI/bge-reranker-v2-m3` (a cross-encoder that re-scores retrieved chunks for relevance).
- **Hybrid retrieval:** combines **BM25** (keyword search) + **vector** (semantic search), fused with **RRF** (Reciprocal Rank Fusion). This is why exact terms like "CP003" and semantic concepts both work.
- **Corrective RAG:** if the top chunk's score is below a threshold (0.35), it automatically retries with an expanded query.
- **Metadata filtering:** each chunk has a `doc_type` (policy / supplier / event / graph / reasoning), so agents only search relevant document types.

6.3 The public API — the methods you will call in eval

Each agent calls one method. **Each returns a formatted context STRING** (not a list of chunks — remember this):

```
retriever.query_for_agent1(category, abc_class, urgency, lead_time_too_late, event_name, demand_t
retriever.query_for_agent2(category, supplier_name, lead_time_too_late, abc_class, urgency)
retriever.query_for_agent3(category, urgency, abc_class, approval_pool)
retriever.query_for_agent4(category, supplier_name, route)
```

Because these return the exact string the agent sees, your retrieval eval checks: *did the right facts land in that string?* (recall) and *did wrong-document facts leak in?* (precision).

7. File-by-file repo walkthrough

```
orca-retail/
├── data/
│   ├── (synthetic data generation scripts)    # builds the raw→staged→curated tables
│   └── scheduler.py                          # simulates real-time alerts; run with --once
├── db/
│   ├── orca.db                              # SQLite database (auto-created on first run)
│   ├── chroma/                              # ChromaDB vector store (created by ingest.py)
│   ├── queries.py                           # all DB read/write functions (the "ontology" req
│   └── pipeline_log.py                       # saves completed pipeline runs to audit table
├── agents/
│   ├── graph.py                             # ★ THE CORE — LangGraph 4-agent pipeline + HITL
│   ├── crew.py                              # CrewAI forecasting crew (⚠ currently failing)
│   ├── llm_factory.py                       # get_llm() — returns Groq/OpenAI/Ollama based or
│   ├── prompts.py                           # the 4 system prompts as ChatPromptTemplate obje
│   └── tools.py                             # @tool functions (the 6 MCP tools' logic)
├── mcp_server/
│   └── server.py                             # MCP server exposing the 6 tools for dynamic dis
├── docs/
│   ├── replenishment_policy.txt              # RAG doc 1
│   ├── supplier_sla.txt                     # RAG doc 2
│   ├── event_playbook.txt                   # RAG doc 3
│   ├── entity_relationships.txt             # RAG doc 4 (GraphRAG)
│   └── agent_reasoning_patterns.txt         # RAG doc 5 (Corrective RAG)
```

```

├── rag/
│   ├── ingest.py                # chunks + embeds docs into ChromaDB
│   └── retriever.py             # ★ hybrid retrieval + rerank + the query_for_agent
├── api/
│   ├── main.py                  # ★ FastAPI app - all endpoints
│   └── models.py                # Pydantic request/response schemas
├── dashboard/
│   ├── app.py                   # ★ Streamlit dashboard (3 tabs)
│   └── api_client.py            # thin HTTP wrapper around the API (UI never calls API)
├── evals/                       # ★ YOUR WORKSPACE
│   ├── golden_dataset.py        # 11 retrieval test cases (already written)
│   └── run_retrieval_eval.py     # retrieval eval runner (already written)
├── Dockerfile.api
├── Dockerfile.dashboard
├── docker-compose.yml
├── requirements.txt              # full deps (local)
├── requirements.api.txt          # slim deps (Render API - no torch/chromadb)
└── .env                         # secrets (GROQ_API_KEY etc.) - NEVER commit

```

Key files explained simply

- **agents/graph.py** — the orchestrator. Defines `AgentState` (the shared notebook that flows through all agents), the 4 agent nodes, the routing logic, and the HITL interrupt. If you read one file, read this.
- **agents/llm_factory.py** — one function `get_llm()` that returns the right LLM object based on `LLM_PROVIDER` in `.env` (groq / openai / ollama). **You will reuse this in your judge eval.**
- **agents/prompts.py** — the 4 system prompts. Each is a `ChatPromptTemplate` with {placeholders} that `graph.py` fills with real data.
- **docs/rag/retriever.py** — the RAG brain. The `query_for_agentN` methods are your eval targets.
- **api/main.py** — exposes `/health`, `/api/v1/alerts`, `/api/v1/pipeline/run`, `/api/v1/pipeline/{id}/state`, `/api/v1/pipeline/{id}/approve`, etc.
- **dashboard/app.py** — 3 tabs: **Command Centre** (alert list + Analyse buttons), **Pipeline Monitor** (live pipeline progress), **HITL Approval** (Approve/Reject for escalated pipelines).

8. Tech stack (with fresher-friendly one-liners)

Tech	What it is	Role in ORCA
LangGraph	Framework for building stateful, multi-step AI agent workflows as a graph	Runs the 4-agent pipeline + HITL pause/resume
LangChain	Toolkit for working with LLMs (prompts, chains)	Prompt templates, LLM calls
CrewAI	Framework for multi-agent "crews" that collaborate	Demand forecasting sub-crew inside Agent 1
RAG	Retrieval-Augmented Generation — fetch relevant docs,	Supplies policy knowledge to

	feed to LLM	agents
ChromaDB	A vector database	Stores the 71 embedded document chunks
Embeddings	Turning text into number vectors for similarity search	nomic / all-MiniLM models
BGE reranker	A model that re-scores search results for relevance	Improves retrieval quality
MCP	Model Context Protocol — Anthropic’s standard for tools to be discovered dynamically	Exposes the 6 data tools
FastAPI	Python web framework for building APIs	The backend the dashboard talks to
Pydantic	Data validation library	Validates every API request/response
Streamlit	Python framework for quick web dashboards	The UI
Docker / docker-compose	Packages the app into portable containers	Runs API + dashboard together
Render	Free cloud hosting platform	Where ORCA is deployed live
Groq	Fast LLM inference provider	Serves llama-3.1-8b-instant (the agent brain)
SQLite	A lightweight file-based database	orca.db
litellm	A library that gives one interface to many LLM providers	Used under the hood by CrewAI

(Section 14 has a fuller glossary of eval-specific terms.)

9. Environment setup — get ORCA running locally (DO THIS DAY 1)

Goal: get the dashboard open in your browser with **API ONLINE** and **RAG: available**, and successfully run one pipeline.

Step 0 — Prerequisites

- Python 3.11 (check: `python --version`)
- Git
- A free **Groq API key** (sign up at groq.com → API Keys). No money needed.

Step 1 — Clone the repo

```
git clone https://github.com/ankitv42/orca-retail.git
```

```
cd orca-retail
```

Step 2 — Create a virtual environment and install dependencies

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
# Mac/Linux:
source .venv/bin/activate

pip install -r requirements.txt
```

If `sentence-transformers` isn't pulled in, install it explicitly: `pip install sentence-transformers`. The RAG retriever needs it.

Step 3 — Create your `.env` file

In the repo root, create a file named `.env`:

```
LLM_PROVIDER=groq
GROQ_API_KEY=your-groq-key-here
GROQ_MODEL=llama-3.1-8b-instant
```

Never commit `.env`. It's in `.gitignore` for a reason.

Step 4 — Build the RAG knowledge base (REQUIRED — RAG won't work without it)

```
python docs/rag/ingest.py --reset
```

This downloads the embedding model (first time only, ~1–2 min) and loads all 71 chunks into `db/chroma/`. Wait for "Ingestion complete".

If you skip this, the dashboard will show `RAG: unavailable` — that's the #1 setup gotcha.

Step 5 — Run the API (terminal 1)

```
uvicorn api.main:app --reload --port 8080
```

Check it: open `http://localhost:8080/health` — you want to see `"rag": "available"`.

Step 6 — Run the dashboard (terminal 2)

```
streamlit run dashboard/app.py
```

The dashboard opens in your browser. Click a SKU's **Analyse** button → watch the pipeline run in the **Pipeline Monitor** tab.

`orca.db` is auto-created on the first pipeline run if it doesn't exist. The `alerts` table is populated by `data/scheduler.py` (run `python data/scheduler.py --once` if the alert list is empty).

Step 7 — Run the eval (terminal, no API key needed)

```
python evals/run_retrieval_eval.py
```

This is your starting point for the eval work (see Section 12).

10. Deployment (Render) — for awareness

ORCA is deployed live on **Render** (free tier) as two web services:

- **API:** <https://orca-retail.onrender.com> (`/health` , `/docs` for Swagger)
- **Dashboard:** <https://orca-dashboard.onrender.com>
- **GitHub:** <https://github.com/ankitv42/orca-retail>

Important deployment fact: On Render, **RAG is intentionally disabled**. The free tier has only 512 MB RAM, which can't fit `torch + sentence-transformers + ChromaDB`. So the API there uses a slim `requirements.api.txt` that excludes those. The agents fall back to LLM knowledge only. This is a **deliberate trade-off, not a bug** — locally (with full requirements) RAG works fully. Do not “fix” RAG on Render; it's by design.

11. Known landmines (READ before debugging anything)

These cost real hours to discover. Don't rediscover them.

11.1 The `C:/lit` / `litellm` path trap (Windows office laptop)

On the office laptop, `litellm` was installed separately with `pip install litellm --target C:/lit`. That folder *also* contains its own copies of `tokenizers` and `huggingface_hub`. The code used `sys.path.insert(0, "C:/lit")`, which put that folder **first** in Python's search path — so the embedding model loaded the **wrong (incompatible) versions** from `C:/lit` and RAG died with `RuntimeError: No embedding model available`.

Fix already applied: change `insert(0, ...)` to `append(...)` in:

- `agents/graph.py` (around line 94)
- `api/main.py` (around line 75)

`append` puts `C:/lit` **last**, so the venv's correct packages win. Recommended clean version for committing:

```
import os
if os.path.exists(r"C:/lit"):
    sys.path.append(r"C:/lit")
```

This is harmless on machines without `C:/lit` (Render, personal laptop).

Also added: a **self-healing** `is_available()` in `retriever.py` that retries init if it failed during startup.

11.2 RAG shows “unavailable” locally

Almost always means **you didn’t run** `python docs/rag/ingest.py --reset` (empty ChromaDB), or `sentence-transformers` isn’t installed in the venv. See Section 9 Step 4.

11.3 The CrewAI / Groq `cache_breakpoint` error (active bug)

CrewAI’s internal LLM call adds a `cache_breakpoint` property to the system message. **Groq’s API rejects it** → the forecasting crew fails **every run** with a 400 error. The code catches this and falls back to a raw-data demand summary, so the pipeline still completes — but the CrewAI crew isn’t actually doing its job. See Issue #1.

11.4 Two laptops, two usernames

History spans an **office laptop** (`C:\Users\ankit.c.kumar.verma\...`) and a **personal laptop** (`C:\Users\anjali\...`). Paths in old logs differ for this reason. You’ll work on whichever machine you’re given — just know the repo is identical.

12. YOUR TASK — the evaluation workstream

This is what you’ll actually build over the fortnight. ORCA evaluation has **three layers**. Layer 1 is started; Layers 2 and 3 are yours.

The mental model (why eval exists)

ORCA makes autonomous money decisions. Without evaluation we’re flying blind — a prompt tweak or a doc change could silently break a decision and we’d never know until a wrong order goes out. Eval is **automated testing for AI behaviour**. On Palantir the platform watched quality for us; in open-source, we build that safety net. That net is your deliverable.

Layer 1 — Retrieval eval (started — your first job is to calibrate it)

Files: `evals/golden_dataset.py`, `evals/run_retrieval_eval.py` **What it does:** calls the real `query_for_agentN()` methods and checks whether the right keywords reached the context (**recall**) and whether wrong-document keywords leaked in (**precision**). Free to run, no API key.

Your first task:

```
python evals/run_retrieval_eval.py
```

Some cases may FAIL on the first run — **not** because retrieval is broken, but because the expected keywords in `golden_dataset.py` were written from memory and may not exactly match the wording in the real `.txt` docs (e.g. expected “approve” but the doc says “authorization”). **Open the 5 docs in `docs/`, read them, and adjust the `must_contain` keyword lists to match the real wording.** This calibration teaches you the whole knowledge base. Aim for ≥70% pass rate with zero keyword leaks.

Pay special attention to the cases marked `"silent_failure": True`. These are traps where the agent could produce a plausible answer even if retrieval missed a key fact (like “Hindustan FMCG does NOT allow expedite”). Catching these is the whole point.

Layer 2 — LLM-as-Judge eval 🛠️ (YOUR MAIN BUILD)

File to create: `evals/run_judge_eval.py` **Concept:** use a **stronger** LLM as a “judge” to score the agents’ decisions 1–5 on quality criteria. Our agents run on `llama-3.1-8b-instant` ; the **judge runs on** `llama-3.3-70b-versatile` (stronger, same Groq key, free).

Interview-grade nuance to understand and write in the README: the judge should ideally be from a *different model family* (e.g. GPT-4o or Claude) to avoid “family bias” — a model tends to rate its own family’s output favourably. We use llama-3.3-70b for cost reasons; note in the docs that the harness is model-agnostic and the judge is a one-line config swap.

Criteria the judge should score (per agent decision):

- 1. **RAG grounding** — did the agent reference the retrieved policy before deciding?
- 2. **HITL accuracy** — was human approval triggered exactly when cost exceeded the auto-approve limit?
- 3. **Scoring-formula correctness** — did Agent 3 apply the formula (Section 4) correctly?
- 4. **Class-A safety** — was Option B correctly never recommended for a Class A SKU?

How to build it (suggested approach):

- Reuse `agents.llm_factory.get_llm()` but point it at `llama-3.3-70b-versatile` (set `GROQ_MODEL` or add a judge-specific factory).
- Feed the judge: the input scenario + the agent’s output + the retrieved context.
- Ask it to return a JSON score (1–5) with a one-line reason per criterion.
- Parse the JSON, aggregate, and print a scorecard.
- Start with the **anchor scenario** (Ajwa Dates ESCALATE; Dish Soap AUTO_EXECUTE) as your first two judged cases.

You can study **DeepEval’s G-Eval** and **RAGAS** as references (these are the industry-standard libraries). You don’t have to use them — a clean hand-rolled judge with `get_llm()` is perfectly fine and arguably more educational — but read their docs so you understand faithfulness, context recall, context precision, and groundedness.

Layer 3 — CI gate 🛠️ (after Layers 1 & 2 are green)

File to create: `.github/workflows/eval_gate.yml` **What it does:** runs the retrieval eval (and optionally the judge eval) automatically on every push to `main` . If scores drop below threshold, the build fails and the merge is blocked. This is **CI/CD** — automated quality gates.

- Start simple: a GitHub Action that checks out the code, installs deps, runs `python evals/run_retrieval_eval.py --ci` , and fails the job on a non-zero exit code.
- The retrieval eval already supports `--ci` (exits 1 if pass rate < 70% or any keyword leaks).

Suggested fortnight plan

Days	Goal

1–2	Get ORCA running locally (Section 9). Read all 5 RAG docs.
3–4	Calibrate Layer 1 retrieval eval to $\geq 70\%$ pass, 0 leaks.
5–8	Build Layer 2 LLM-as-judge (<code>run_judge_eval.py</code>). Judge the anchor scenarios first, then expand.
9–10	Build Layer 3 CI gate (<code>eval_gate.yml</code>).
11–12	Write findings into <code>docs/eval_findings.md</code> . Tidy, commit, document.
Buffer	Investigate any silent-failure traps that fail; tune docs/retriever if needed.

Commit often, with clear messages. Push to a **branch**, not directly to `main` , and open a PR so Ankit can review on return.

13. ISSUE TRACKER (everything pending — READ THIS)

Ordered roughly by priority. Items marked **[eval]** are within your workstream; others are for awareness / if you have spare time.

#	Issue	Severity	Status	Notes
1	CrewAI <code>cache_breakpoint</code> Groq error — forecasting crew fails every run, falls back to raw-data summary	HIGH	Open	CrewAI adds <code>cache_breakpoint</code> to the system msg; Groq rejects it. Likely a litellm version pin or a CrewAI LLM config flag to disable prompt caching. Undercuts the “3-agent CrewAI crew” portfolio claim.
2	Layer 2 LLM-as-judge eval not built	HIGH	[eval] — your task	See Section 12.
3	Layer 3 CI/CD eval gate not built	MEDIUM	[eval] — your task	GitHub Actions. See Section 12.
4	Unit tests (pytest) missing	HIGH	Open	No <code>tests/</code> suite yet. <code>tools.py</code> and the API endpoints need unit tests with mocked LLM calls. Strong engineering signal if added.
5	LangSmith tracing not added	MEDIUM	Open	<code>LANGCHAIN_TRACING_V2=true</code> + API key would give per-node observability (latency, tokens, cost). Not started.
6	Layer 1 retrieval eval needs calibration	MEDIUM	[eval] — your task	Keyword lists written from memory; tune to real docs. See Section 12.

7	Stale startup log line in <code>api/main.py</code> (~line 156) prints <code>RAG: unavailable</code> (Windows limitation) even when RAG is available	LOW	Open	Cosmetic only — the <code>/health</code> endpoint reports correctly. Just an old log string.
8	<code>/api/v1/alerts</code> returns null fields — <code>store_id</code> , <code>stock_status</code> , <code>current_stock</code> , <code>days_of_cover</code> , <code>risk_score</code> come back null	LOW	Accepted	Root cause: <code>get_critical_alerts()</code> joins at SKU level, not store level. Accepted because the pipeline fetches fresh store data when Analyse is clicked. Not a demo blocker.
9	Dashboard auto-refresh was unreliable; reworked to always-on 3s polling	LOW	Mostly resolved	If it regresses, the polling lives in <code>dashboard/app.py</code> . Earlier attempts with <code>streamlit-autorefresh</code> and JS were flaky.
10	First-SKU bug — the very first SKU clicked sometimes errored while 2nd/3rd worked	LOW	Intermittent	Seen during Docker testing. Re-verify locally; may already be resolved.
11	ADR documents not written	MEDIUM	Open	<code>docs/adr/</code> with 5 Architecture Decision Records (LangGraph vs LangChain, ChromaDB vs Pinecone, embeddings choice, MCP vs hardcoded tools, Cloud Run vs K8s). For system-design interview prep.
12	<code>C:/lit</code> path lines should be made conditional before committing	LOW	Recommended	Use the <code>if os.path.exists(r"C:/lit")</code> guard (Section 11.1) so the line is clean across all machines.
13	Render RAG disabled	N/A	By design	Not a bug — slim requirements for 512 MB free tier. Don't "fix".
14	LeetCode daily practice	N/A	Ankit's personal habit	Not part of the repo; ignore for eval work.

14. Glossary (whenever you're stuck)

- **Agent** — an LLM given a specific role, prompt, and tools, that performs one reasoning step.
- **Multi-agent pipeline** — several agents chained so each one's output feeds the next.
- **HITL (Human-In-The-Loop)** — the system pauses for a human decision before acting.
- **LangGraph node / edge** — a node is one step (an agent); edges connect steps. The graph defines the

flow.

- **AgentState** — a shared dictionary (a “notebook”) that flows through every node carrying all results.
- **Checkpoint** — saves the graph’s state so it can pause and resume (how HITL works).
- **RAG (Retrieval-Augmented Generation)** — fetch relevant documents and put them in the prompt so the LLM answers from real knowledge, not just memory.
- **Chunk** — a small slice of a document stored in the vector DB.
- **Embedding** — a numeric vector representing text meaning; similar texts have similar vectors.
- **Vector search** — finding chunks whose embeddings are closest to the query embedding.
- **BM25** — a classic keyword-based search algorithm (good for exact terms like “CP003”).
- **Hybrid retrieval** — combining vector + BM25 search.
- **RRF (Reciprocal Rank Fusion)** — a formula to merge two ranked lists into one.
- **Reranker** — a model that re-scores retrieved chunks for relevance to the query.
- **Corrective RAG** — if retrieval looks weak, automatically retry with a better query.
- **MCP (Model Context Protocol)** — a standard that lets agents discover and call tools dynamically.
- **Tool** — a Python function the agent can call to fetch data or take an action.
- **Eval (evaluation)** — automated testing of AI behaviour quality.
- **Golden dataset** — a hand-written set of test cases with known-correct expectations.
- **LLM-as-judge** — using a (stronger) LLM to grade another LLM’s output.
- **G-Eval** — a specific LLM-as-judge technique (chain-of-thought scoring on custom criteria).
- **Faithfulness** — does the answer come only from the provided context (no hallucination)?
- **Groundedness** — sentence-level version of faithfulness (which exact sentence is supported?).
- **Context recall** — did retrieval fetch *all* the relevant info? (the “silent failure” metric)
- **Context precision** — was the retrieved info actually relevant (not noisy)?
- **Silent failure** — when the system looks healthy on one metric but is quietly broken on another (e.g. high faithfulness, low recall).
- **CI/CD** — Continuous Integration / Deployment; automated checks that run on every code push.
- **CI gate** — a check that blocks a merge if quality drops.
- **ABC class** — product importance tiers: A (most important, strict rules), B, C.
- **Capital pool** — a budget bucket (CP001, CP003...) with an auto-approve limit and a pressure flag.
- **Lead time** — days from placing an order to receiving stock.
- **Expedite** — faster (air freight) delivery at a premium cost = Option C.

- **AUTO_EXECUTE / ESCALATE / SUSPEND** — the three pipeline outcomes (Section 4).
-

15. Where to find things / contacts

- **Repo:** <https://github.com/ankitv42/orca-retail>
 - **Live API:** <https://orca-retail.onrender.com> (and `/docs` for the Swagger UI)
 - **Live dashboard:** <https://orca-dashboard.onrender.com>
 - **Your workspace in the repo:** the `evals/` folder.
 - **Ankit:** travelling (USA) for ~2 weeks. Batch your questions; push work to a branch + PR for review on return. For anything that blocks you completely, note it in the issue tracker above and move to the next task.
-

Final note for you

You've joined at a genuinely good moment — the build is done, it's deployed and working, and your job (evaluation) is the part that separates a "demo" from a "production-grade" system. Take your time with Section 9 (get it running) and Section 6 (understand RAG) before writing any eval code. Read the five `.txt` docs end to end — they *are* the business logic. Everything else will click from there.

Good luck. Commit often. Ask good questions.

Appendix A — File Purposes & Data Flow Traces

Companion to the main KT document. This appendix does two things:

1. **Part 1** — every code file: what it does, why it matters, its role in ORCA, and an importance rating.
2. **Part 2** — three concrete end-to-end traces: you click **Analyse**, and we follow the data through every file, function, and decision until the result lands back on your screen.

Read Part 1 to know *what each piece is*. Read Part 2 to see *how they work together*. Part 2 is where it truly clicks.

PART 1 — File-by-file purpose & importance

Importance legend: ★★★★★ = core, you must understand it · ★★★★ = important · ★ = supporting/reference

Data & Database layer

`data/` (synthetic data generation scripts)

- **What it does:** Generates the fake-but-realistic retail dataset — stores, SKUs, suppliers, capital pools, events, and inventory positions — and writes them into the SQLite database in three layers (raw → staged → curated).
- **Why it matters:** Without it there's no data for anything to run on. It's the "factory" that creates ORCA's world.
- **Role in ORCA:** One-time setup. Models the real UAE retail dataset from the original Palantir RCC project, but synthetic so it's shareable.
- **Importance:** ★ (run once, rarely touched)

`data/scheduler.py`

- **What it does:** Simulates the real-time monitoring that, in a live system, would run continuously. Scans inventory positions and marks SKUs as `CRITICAL` or `AT_RISK` in the `alerts` table. Run with `--once` for a single pass.
- **Why it matters:** It's what populates the alert list the dashboard shows. No scheduler run = empty Command Centre.
- **Role in ORCA:** The "trigger" source. In Palantir, the Ontology auto-fired when status changed; here `scheduler.py` plays that role.
- **Importance:** ★★

`db/queries.py`

- **What it does:** Every raw database read/write as a plain Python function (SQL → dict). E.g.
`get_all_positions_for_sku()` , `get_supplier_for_sku()` , `get_capital_pool()` ,
`writeback_reorder_for_all_positions()` .
- **Why it matters:** It's the single, clean data-access layer. Nothing else writes SQL. This is the **open-source replacement for Palantir's Ontology object queries**.
- **Role in ORCA:** The foundation every tool and agent ultimately depends on for facts.
- **Importance:** ★★★

`db/pipeline_log.py`

- **What it does:** Creates and writes to the `pipeline_log` audit table — saves every completed pipeline run (status, all agent outputs, briefing).
- **Why it matters:** Audit trail. You can look back at what every pipeline decided and why. Also feeds the dashboard's pipeline history.
- **Role in ORCA:** Persistence + auditability (replaces Palantir's writeback object for state).
- **Importance:** ★★

`db/orca.db`

- **What it does:** The actual SQLite database file holding all tables.
 - **Why it matters:** It's the single source of truth for structured facts.
 - **Role in ORCA:** The database. Auto-created on first run; rebuilt by the data scripts.
 - **Importance:** ★★☆☆ (it's the data — but you don't edit it by hand)
-

Agent / AI layer

agents/graph.py — THE CORE FILE

- **What it does:** Defines the entire LangGraph pipeline: the shared `AgentState`, the four agent nodes (`agent1_node ... agent3_node`), the pure-Python `route_node`, the three terminal nodes (`hitl_node`, `execute_node`, `suspend_node`), the `save_node`, all the edges wiring them together, and the HITL interrupt. Exposes `run_pipeline()`, `resume_pipeline()`, and `get_pipeline_state()`.
- **Why it matters:** This is the orchestrator. Everything else is a dependency this file pulls in. If you read one file end to end, read this one.
- **Role in ORCA:** The brain stem — turns a single SKU trigger into a full reorder decision.
- **Importance:** ★★☆☆
- **Key internals to know:**
 - `AgentState` (TypedDict) — the shared "notebook" carrying every agent's inputs and outputs down the chain.
 - `interrupt_before=["execute_node"]` — the HITL pause.
 - `MemorySaver` checkpointer keyed by `thread_id = pipeline_id` — how a paused pipeline resumes.
 - It initialises the RAG retriever and is where the `C:/lit` path line lives (Issue #12).

agents/llm_factory.py

- **What it does:** One function, `get_llm()`, returns the correct LLM object based on `LLM_PROVIDER` in `.env` (groq / openai / ollama). Plus `get_provider_name()` and `get_model_name()` for logging.
- **Why it matters:** Single switch point. Change one line in `.env` and the whole system swaps LLM provider. **You will reuse this in your judge eval** (point it at `llama-3.3-70b-versatile`).
- **Role in ORCA:** Provider abstraction — proves ORCA isn't locked to one vendor (a real interview question).
- **Importance:** ★★☆☆ (especially for you — eval reuses it)

agents/prompts.py

- **What it does:** Holds the four agents' system prompts as LangChain `ChatPromptTemplate` objects with {placeholders}. Exposes a `PROMPTS` dict (`PROMPTS["agent1"]` etc.).
- **Why it matters:** This is where the *business rules in natural language* live — the actual instructions that

make each agent behave. These were translated directly from the Palantir RCC agent prompts.

- **Role in ORCA:** The “personality and rules” of each agent. `graph.py` fills the placeholders with real data and sends them to the LLM.
- **Importance:** ★★☆☆ (the agent behaviour you evaluate is shaped here)

`agents/tools.py`

- **What it does:** The six `@tool`-decorated functions the agents use to fetch structured data. Each wraps one or more `queries.py` functions and sometimes adds derived fields.
- **Why it matters:** These are the agent’s “hands” — how it reaches into the database.
- **Role in ORCA:** The tool layer. Also exposed via MCP (see `server.py`).
- **Importance:** ★★☆☆
- **The six tools (and what each returns):**

Tool	Wraps (<code>queries.py</code>)	Returns / adds
<code>check_inventory_positions(sku_id)</code>	<code>get_all_positions_for_sku()</code>	<code>positions + critical_count , at_risk_count , total_current_stock , total_projected_shortfall</code>
<code>get_sku_info(sku_id)</code>	<code>get_sku_details()</code>	SKU master (category, abc_class, margin)
<code>get_supplier_info(sku_id)</code>	<code>get_supplier_for_sku()</code>	supplier (lead time, allows_expedite , premium, contact)
<code>get_demand_velocity(sku_id)</code>	<code>get_sales_velocity()</code>	recent sales rate
<code>check_active_events(category)</code>	<code>get_active_events_for_category()</code>	events + events_found count
<code>check_capital_budgets(pool_id?)</code>	<code>get_capital_pool() or get_all_capital_pools()</code>	pool budget, auto_approve_limit_aed , pool_pressure_flag

`agents/crew.py`

- **What it does:** Defines a **CrewAI** multi-agent forecasting “crew” (a Senior Data Analyst agent + supporting roles) that Agent 1 calls for a deeper demand read. Uses `llama-3.3-70b-versatile`.
- **Why it matters:** Demonstrates a *second* multi-agent pattern (role-based CrewAI vs graph-based LangGraph) — a strong portfolio point.
- **Role in ORCA:** Sub-crew inside Agent 1’s demand analysis.
- **Importance:** ★★☆☆ — **but currently broken** (Issue #1: Groq rejects CrewAI’s `cache_breakpoint` ; the crew fails and Agent 1 falls back to a raw-data summary). The pipeline still completes via the fallback.

MCP layer

`mcp_server/server.py`

- **What it does:** Runs an **MCP (Model Context Protocol) server** that exposes the six tools so the agent can *discover them dynamically at runtime* instead of importing them as hardcoded functions.
- **Why it matters:** MCP is the same open standard used by Claude Desktop and Cursor. "I exposed my tools via an MCP server" is a current, in-demand interview line.
- **Role in ORCA:** The dynamic tool-discovery layer between the agent and the data tools. The graph connects to it via `langchain-mcp-adapters`.
- **Importance:** ★★

RAG layer (your evaluation focus)

`docs/*.txt` (the 5 knowledge documents)

- **What it does:** Hold the policy knowledge the database can't express — ordering rules, supplier SLAs, event playbooks, entity relationships, agent reasoning patterns.
- **Why it matters:** They *are* the business logic in prose. **Read all five before writing eval code** — your golden-dataset keywords must match their real wording.
- **Role in ORCA:** The RAG knowledge base (71 chunks once ingested).
- **Importance:** ★★★ (for you specifically)

`docs/rag/ingest.py`

- **What it does:** Reads the 5 docs, splits them into semantic chunks, attaches metadata (`doc_type`), embeds them with the embedding model, and stores them in ChromaDB at `db/chroma/`. Run with `--reset` to rebuild.
- **Why it matters:** No ingest = empty vector store = `RAG: unavailable`. This is the #1 setup gotcha.
- **Role in ORCA:** Builds the searchable knowledge base.
- **Importance:** ★★★

`docs/rag/retriever.py` — your main eval target

- **What it does:** The retrieval brain. Hybrid search (BM25 + vector + RRF), BGE reranking, corrective RAG, metadata filtering. Exposes `query_for_agent1..4()` which each return a **formatted context string**, plus `is_available()`.
- **Why it matters:** This decides *what policy knowledge each agent sees*. Your retrieval eval calls these exact methods.
- **Role in ORCA:** Connects the knowledge base to the agents.

- **Importance:** 🌟🌟🌟 (for you specifically)
- **Note:** Contains the self-healing `is_available()` fix (Issue #11.1 in main KT).

API layer

`api/main.py`

- **What it does:** The FastAPI backend. Key endpoints:

Endpoint	Method	Purpose
<code>/health</code>	GET	Liveness + readiness (db, rag, llm, mcp status)
<code>/api/v1/alerts</code>	GET	Current critical/at-risk SKU list (dashboard feed)
<code>/api/v1/pipeline/run</code>	POST	Launch a pipeline run as a background task , returns 202 + <code>pipeline_id</code> immediately
<code>/api/v1/pipeline/{id}/state</code>	GET	Current state of a running pipeline (polled every 3s by the dashboard)
<code>/api/v1/pipeline/{id}/approve</code>	POST	Human approve/reject → resumes the paused pipeline
<code>/api/v1/pipeline/{id}/briefing</code>	GET	The HITL briefing text
<code>/api/v1/pipelines</code>	GET	List of pipeline runs (history)

- **Why it matters:** It's the bridge between the UI and the agent pipeline. The **background-task pattern** is crucial: a pipeline takes ~60–90s, so the API kicks it off in the background and returns instantly, then the dashboard polls for progress (otherwise the UI would freeze).
- **Role in ORCA:** The backend server.
- **Importance:** 🌟🌟🌟

`api/models.py`

- **What it does:** Pydantic schemas for every request and response (e.g. `RunPipelineRequest`, `PipelineStateResponse`, `HealthResponse`, plus enums like `PipelineStatus`, `RouteDecision`, `Urgency`).
 - **Why it matters:** Guarantees the shape of all data in and out of the API. Bad input → 422 error before any logic runs.
 - **Role in ORCA:** The API contract.
 - **Importance:** 🌟🌟
 - **Note:** `order_qty` is `Optional[float]` on purpose — the LLM sometimes returns decimals like `597.8`, which crashed an earlier `int`-typed version.
-

Dashboard layer

`dashboard/app.py`

- **What it does:** The Streamlit UI. Three tabs: **Command Centre** (alert list + Analyse buttons + KPI cards), **Pipeline Monitor** (live progress of the running pipeline, auto-refreshing every 3s), **HITL Approval** (Approve/Reject buttons for escalated pipelines).
- **Why it matters:** It's what a human actually sees and clicks. The open-source rebuild of the Palantir Workshop dashboard.
- **Role in ORCA:** The human interface.
- **Importance:** ★★☆☆
- **Note:** Tracks processed SKUs in `session_state` so a SKU's Analyse button becomes  ANALYSED after one click.

`dashboard/api_client.py`

- **What it does:** A thin HTTP wrapper around every API call (`get_alerts()` , `run_pipeline()` , `get_pipeline_state()` , `approve_pipeline()` ...). Returns plain dicts; handles errors so the UI never crashes.
 - **Why it matters:** Keeps `app.py` pure UI logic — it never touches HTTP directly. Clean separation.
 - **Role in ORCA:** UI ↔ API glue.
 - **Importance:** ★★☆☆
 - **Note:** `BASE_URL` comes from the `API_BASE_URL` env var (so local vs Render just changes one variable).
-

Eval layer (yours)

`evals/golden_dataset.py`

- **What it does:** 11 hand-written retrieval test cases (real data, real `query_for_agentN` kwargs, expected keywords, forbidden keywords, silent-failure flags).
- **Importance:** ★★☆☆ (your starting point — calibrate it)

`evals/run_retrieval_eval.py`

- **What it does:** Runs the golden cases against the real retriever, scores keyword coverage (recall) and leakage (precision), reports a scorecard, supports `--ci`.
- **Importance:** ★★☆☆ (your Layer 1)

`evals/run_judge_eval.py` **(to be created by you) — Layer 2**

`.github/workflows/eval_gate.yml` **(to be created by you) — Layer 3**

Infra / config

File	What it does	Importance
Dockerfile.api	Builds the API container (uses slim requirements.api.txt)	★
Dockerfile.dashboard	Builds the dashboard container	★
docker-compose.yml	Runs API + dashboard together with one command	★★
requirements.txt	Full local dependencies (includes torch, chromadb, sentence-transformers)	★★
requirements.api.txt	Slim deps for Render API (excludes heavy ML libs to fit 512 MB)	★★
.env	Secrets (GROQ_API_KEY etc.) — never commit	★★★

PART 2 — End-to-end data flow traces

Now we follow the data. Three scenarios, from the click to the result. Trace #1 is the full walkthrough; #2 and #3 show how the *same* pipeline produces *different* outcomes.

TRACE #1 — “Analyse” → AUTO_EXECUTE (the simplest happy path)

Scenario: A cheap Grocery item, low cost, plenty of budget. No human needed.

The 12 steps

```
[1] USER clicks "Analyse" on SKU00097 (Dish Soap) in Command Centre tab
    | dashboard/app.py
    ▼
[2] app.py calls api_client.run_pipeline("SKU00097", "STR0001")
    | dashboard/api_client.py → HTTP POST
    ▼
[3] POST /api/v1/pipeline/run hits api/main.py
    | validates body with Pydantic (RunPipelineRequest)
    | starts the pipeline as a BACKGROUND TASK
    | returns 202 + {pipeline_id: "PIPE_SKU00097_2026-06-01"} IMMEDIATELY
    ▼
[4] Dashboard switches to Pipeline Monitor tab and starts POLLING
    every 3s: GET /api/v1/pipeline/PIPE_SKU00097.../state
    (this is why the UI never freezes during the ~60s run)
    ▼
    — meanwhile, in the background task: agents/graph.py run_pipeline() —
    ▼
[5] AGENT 1 node (Demand Intelligence)
    |— pre-fetches data via tools.py:
```



```

|     check_inventory_positions("SKU00097") → tools.py → queries.py → orca.db
|     get_sku_info("SKU00097")             → category="Grocery", abc_class="C"
|     get_demand_velocity("SKU00097")
|     check_active_events("Grocery")
├ fetches RAG context:
|     retriever.query_for_agent1(category="Grocery", abc_class="C",
|                               urgency=..., lead_time_too_late=...)
|     → returns a policy/event context STRING
├ (tries CrewAI crew → FAILS on Groq cache_breakpoint → falls back to raw summary)
├ calls LLM (Groq llama-3.1-8b) with prompt + data + RAG context
├ writes demand_summary into AgentState
|     urgency = MEDIUM, lead_time_too_late = False
▼
[6] AGENT 2 node (Supply Replenishment)
|     get_supplier_info("SKU00097"), get_tier1_stores_for_sku()
|     retriever.query_for_agent2(category="Grocery", supplier_name=..., abc_class="C")
|     LLM builds 3 options (A=standard, B=partial, C=expedite)
├ writes options_package into AgentState (recommended = Option A)
▼
[7] AGENT 3 node (Capital Allocation)
|     check_capital_budgets("CP001"), check_capital_budgets("CP003")
|     retriever.query_for_agent3(category="Grocery", urgency="MEDIUM", approval_pool="CP001")
|     LLM scores options with the formula:
|         budget_score, availability_score, margin_score, lead_time_penalty
├ cost = AED 817 < CP001 auto_approve_limit (e.g. 50,000)
├ writes capital_decision: approval_required = FALSE
▼
[8] ROUTE node (pure Python — NO LLM)
    pool_pressure != HIGH AND approval_required == FALSE
    → route = "AUTO_EXECUTE"
▼
[9] HITL interrupt fires (interrupt_before=["execute_node"])
    but for AUTO_EXECUTE the system clears it immediately — no human wait
▼
[10] EXECUTE node
    writeback_reorder_for_all_positions("SKU00097") → queries.py → orca.db
    sets reorder_triggered = "Yes" on all affected positions
    writes an AUTO-EXECUTED confirmation briefing
▼
[11] SAVE node
    save_pipeline_run(...) → db/pipeline_log.py → orca.db
    final_status = AUTO_EXECUTED
▼
[12] Dashboard's next 3s poll sees status = AUTO_EXECUTED
    → shows the completed result + briefing. Done. No human action.

```

Key teaching point: Notice steps 5–7 always do the *same two things* — pull **structured data** (tools → queries → DB) and **policy knowledge** (RAG retriever). That double-fetch pattern is the heartbeat of every agent. Your eval measures the RAG half of that heartbeat.

TRACE #2 — “Analyse” → ESCALATE (the HITL path — the important one)

Figure 2 — the ESCALATE / human-in-the-loop flow for the Ajwa Dates Ramadan scenario.

Scenario: Ajwa Dates during Ramadan. Expedite air-freight is needed, and it's expensive enough to need human approval.

Steps 1–7 are the same shape as Trace #1, but the *values* differ:

```
[5] AGENT 1: 9 critical stores + Ramadan event 18 days away,
             supplier lead time 26.75 days > 18 days
             → urgency = CRITICAL, lead_time_too_late = TRUE
             → RAG context includes Ramadan uplift + expedite rules

[6] AGENT 2: lead_time_too_late = TRUE → Option A (standard) marked INSUFFICIENT
             → Option C (expedite) recommended
             (RAG confirms this supplier allows_expedite = Yes)

[7] AGENT 3: Option C cost = AED 108,909
             CP003 auto_approve_limit = AED 20,000
             108,909 > 20,000 → approval_required = TRUE

[8] ROUTE:   approval_required == TRUE → route = "ESCALATE"

[9] HITL node (runs BEFORE the interrupt):
     retriever.query_for_agent4(category="Grocery", supplier_name=..., route="ESCALATE")
     LLM writes a BRIEFING: cost, pool, supplier contact, why approval needed
     final_status = ESCALATED
     ▼
      PIPELINE PAUSES HERE (interrupt_before=["execute_node"])
     the MemorySaver checkpoint stores the full state under thread_id = pipeline_id
     ▼
[10] Dashboard poll sees status = ESCALATED
     → HITL Approval tab shows the briefing + Approve / Reject buttons
     ▼
      ... waits for a HUMAN (could be seconds or hours — state is saved) ...
     ▼
[11] HUMAN clicks Approve
     → app.py → api_client.approve_pipeline(pipeline_id, reviewer)
     → POST /api/v1/pipeline/{id}/approve → graph.resume_pipeline(id, approved=True)
     ▼
[12] Pipeline RESUMES from the checkpoint → EXECUTE node runs
     writeback_reorder_for_all_positions(...) → order placed
     SAVE node → final_status = EXECUTED_AFTER_APPROVAL
     ▼
[13] Dashboard poll shows the completed, human-approved result.
```

Key teaching point: The pipeline literally *stops mid-execution* and survives the wait because LangGraph's checkpointers saved every variable. When the human approves hours later, it picks up exactly where it left off. **That pause/resume is the single most important concept in ORCA** — it's the open-source rebuild of Palantir's HITL approval, and it's the answer to the interview question "how do you implement human-in-the-loop?"

TRACE #3 — “Analyse” → SUSPEND (the budget-blocked path)

Scenario: A needed order, but the capital pool is out of money this period.

```
[5-7] Agents run normally and produce a recommended option with a cost.

[7] AGENT 3:  the relevant pool's pool_pressure_flag == HIGH
              (budget too tight to take on more commitments now)

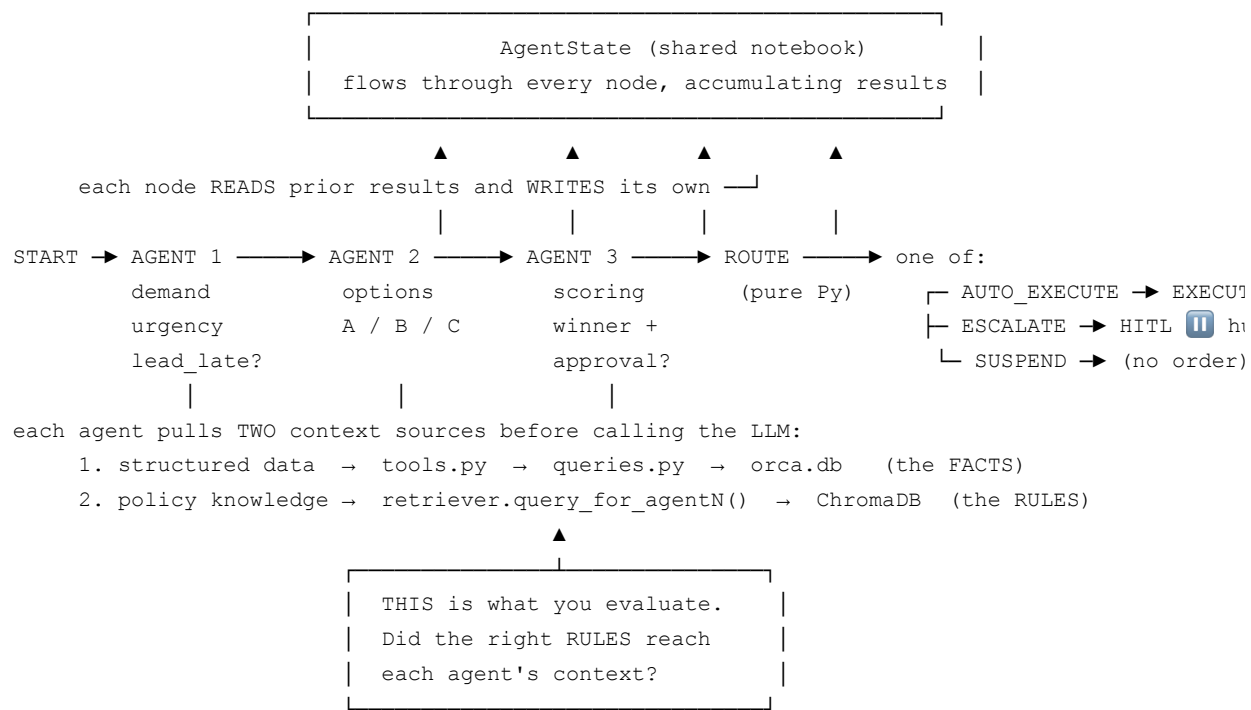
[8] ROUTE:    pool_pressure == HIGH → route = "SUSPEND"
              (this check comes FIRST – it beats both AUTO_EXECUTE and ESCALATE)

[9] SUSPEND node:
    NO order is written. NO money committed.
    Writes a briefing: "SUSPENDED – pool pressure HIGH, order cannot be placed.
    Alert will repeat when pool pressure reduces."
    final_status = SUSPENDED
    ▼

[10] SAVE node → logs it.  Dashboard shows SUSPENDED. No human action, no order.
```

Key teaching point: Routing priority is **SUSPEND first, then AUTO_EXECUTE, then ESCALATE**. A high-pressure pool blocks everything — the system refuses to overcommit a strained budget even for an urgent item. This is a deliberate financial safety rule, and it’s exactly the kind of thing your **judge eval** should verify (was SUSPEND chosen correctly when pressure was HIGH?).

The one diagram that ties it all together



How this connects to your eval work

Now that you've seen the flow, your evaluation makes concrete sense:

- **Layer 1 (retrieval eval)** tests the bottom arrow above — for a given agent + situation, does `query_for_agentN()` return a context string containing the right RULES? (e.g. for Trace #2, does Agent 2's context actually contain "expedite" rules and the supplier's expedite permission?)
- **Layer 2 (LLM-as-judge)** tests the *decisions* the agents made with those rules — did Agent 3 apply the scoring formula right? Did ROUTE correctly pick ESCALATE in Trace #2 and SUSPEND in Trace #3? Did Class A never get Option B?
- **Layer 3 (CI gate)** runs Layers 1–2 automatically on every code push, so nobody can silently break a decision.

Re-read Trace #2 once more before you start — the ESCALATE/HITL path is the scenario your judge eval should anchor on first.